

Hands-on Activity 8.1 Classless Objectives, Classes, and Class-Based Approach in JavaScript	
Course Code: CPE025 B	Program: BSCPE
Course Title: Software Design 2	Date Performed: 03/11/2026
Section: CPE32S4	Date Submitted: 03/11/2026
Name: Bautista, Jhon Hendricks	Instructor: Engr. Neal Barton James Matira
1. Discussion	
JavaScript objects serve as the foundational building blocks for bundling related data into lightweight, memory-efficient configuration blocks without the need for a formal class structure. By utilizing the object literal syntax, developers can quickly wrap comma-separated key-value pairs in curly braces to create flexible data models where properties can be added, modified, or deleted at any time. Accessing these properties is achieved through clean dot notation for standard identifiers or bracket notation when handling dynamic keys and spaces, while more advanced operations like deep cloning via <code>structuredClone()</code> ensure that independent copies are created without shared references. While literals handle simple data, JavaScript classes provide a more structured approach by offering syntactic sugar over traditional prototype-based inheritance. Classes bundle constructors, methods, and properties into a single block, automatically enforcing strict mode and requiring the <code>new</code> keyword for instantiation to prevent accidental globals. This system supports sophisticated patterns such as inheritance through the <code>extends</code> keyword, allowing subclasses to override methods while still accessing parent functionality via <code>super()</code>, and the use of static members for utility functions that belong to the class itself rather than individual instances.	
2. Materials and Equipment	
What materials did you use? Explain in detail.	
3. Procedure	
Create a js file or code on a JS simulator Scenario A logistics company requires a basic system to track delivery vehicles and their current deployment status. The programmer must implement a class to represent individual vehicles and a management class to oversee the fleet. Step-by-Step Procedures 1. Create a Vehicle class. The constructor requires two arguments for id and type. It must initialize an <code>isDeployed</code> property to false. 2. Create a FleetManager class. It must contain a vehicles array to store the fleet. 3. Add an <code>addVehicle</code> method to FleetManager that accepts a Vehicle object and pushes it into the vehicles array. 4. Add a <code>deployVehicle</code> method to FleetManager that accepts a vehicle id. It must find the matching vehicle in the array and change its <code>isDeployed</code> status to true. If the vehicle is not found, it should return undefined. 5. Add a <code>getAvailableVehicles</code> method to FleetManager that returns an array of all vehicles where <code>isDeployed</code> is false.	
4. Output	

JavaScript

```
class Vehicle {
  constructor(id, type) {
    this.id = id;           // Unique ID
    this.type = type;       // Type of vehicle
    this.isDeployed = false; // Deployment
  }
}

class FleetManager {
  constructor() {
    this.vehicles = [];

    addVehicle(vehicle) {
      if (vehicle instanceof Vehicle) {
        this.vehicles.push(vehicle);
      } else {
        console.warn("Only Vehicle instances can be added!");
      }
    }

    deployVehicle(id) {
      const vehicle = this.vehicles.find(v => v.id === id);
      if (vehicle) {
        vehicle.isDeployed = true;
        return vehicle;
      } else {
        return undefined;
      }
    }

    getAvailableVehicles() {
      return this.vehicles.filter(v => !v.isDeployed);
    }
  }

  // Test Entity Instantiation
  console.log("*** Test Entity Instantiation***");
  const testVehicle = new Vehicle("V01", "Truck");
  console.log(testVehicle);
  // Expect Vehicle object with id: V01, type: Truck, isDeployed: false
  // Test Composition and Insertion
  console.log("\n*** Test Composition and Insertion***");
  const fleet = new FleetManager();
  fleet.addVehicle(testVehicle);
}
```

```

fleet.addVehicle(new Vehicle("V02", "Van"));
fleet.addVehicle(new Vehicle("V03", "Drone"));
console.log(fleet.vehicles.length);
// Expect: 3
// Test State Mutation
console.log("\n*** Test State Mutation ***");
fleet.deployVehicle("V02");
const deployedVehicle = fleet.vehicles.find(v => v.id === "V02");
console.log(deployedVehicle.isDeployed);
// Expect: true
// Test Data Filtering
console.log("\n*** Test Data Filtering ***");
const available = fleet.getAvailableVehicles();
console.log(available.length);
// Expect: 2
console.log(available.map(v => v.id));
// Expect: [ V01, V03 ]

```

Figure 1. Procedure Program

The program in figure 1 shows the usage of a class constructor in javascript. This program simulates the fleet manager wherein we can create objects which is the vehicles that will have the id and type parameter then each created object has the id, type, isDeployed property. After that is another class is created which is for containing the vehicle objects created in a array. For this class, it has several methods which mainly for manipulating the vehicle array like adding, deploying, getting available vehicles. These methods utilize array manipulation and comparison operators in order to perform their logic. Lastly is printing the output of the methods through a console.log statement.

```

*** Test Entity Instantiation***                                procedure.js:43
  ▼ Vehicle 1                                                    procedure.js:45
    id: "V01"
    isDeployed: false
    type: "Truck"
    ▶ [[Prototype]]: Object
*** Test Composition and Insertion***                             procedure.js:48
3                                                                    procedure.js:53
                                                                    procedure.js:56
*** Test State Mutation ***
true                                                                procedure.js:59
                                                                    procedure.js:62
*** Test Data Filtering ***
2                                                                    procedure.js:64
  ▼ Array(2) 1                                                    procedure.js:66
    0: "V01"
    1: "V03"
    length: 2
    ▶ [[Prototype]]: Array(0)

```

Figure 2. Output of the Program

Based on the output of figure 2 the program was correct since the expected output was achieved. This shows the program has proper entity instantiation and the usage of the methods were successful. The first test It was able to correctly call the added vehicle.

Then after adding new vehicles the program was able to correctly get the number of vehicles. Then next is testing the value property of the vehicle id and it was successful.

Lastly is the printing of the available vehicles and it was able to do this correctly.

5. Supplementary Activity

The screenshot displays two browser windows. The left window shows a Google account login overlay for 'qjhtbautista@tip.edu.ph' with options to 'Add account' or 'Sign out'. The right window shows the '1.12. Module 1 Completion - Module Test' results for a JavaScript quiz. A large green arc indicates a score of 95%. The text states: 'You have scored 95%. Congratulations, you have passed the exam. You have used 1 of 1 attempt(s). A link to 'Review Assessment' is provided at the bottom.

MODULE 1 QUIZ SCORE

This screenshot shows two browser windows. The left window is identical to the one above, showing the Google account login overlay. The right window shows the '2.8. Module 2 Completion - Module Test' results. A large green arc indicates a score of 100%. The text states: 'You have scored 100%. Congratulations, you have passed the exam. You have used 1 of 1 attempt(s). A link to 'Review Assessment' is provided at the bottom. On the left side of the right window, a 'Course Outline' is visible, listing sections 2.1 through 2.7, with section 2.8 marked as completed.

MODULE 2 QUIZ SCORE

CONCLUSION

After performing this activity I was able to learn about the uses of classes and methods in javascript. In this activity I was able to utilize the OOP fundamentals in javascript and how it is implement in a sample program. For this activity it mainly focused on using the class constructor and methods for simulating a Fleet manager program. In this program I was able to utilize classes to create the vehicle object and reuse

this to create more vehicles and also implement methods for different manipulation tasks needed on the procedure. Overall this activity helped me understand the implementation of OOP principles in javascript.